



THE MANUAL YOU NEVER OPENED

# 17 Claude guides that put you ahead of **95%** of users

Every cookbook, prompting guide  
and course Anthropic publishes.



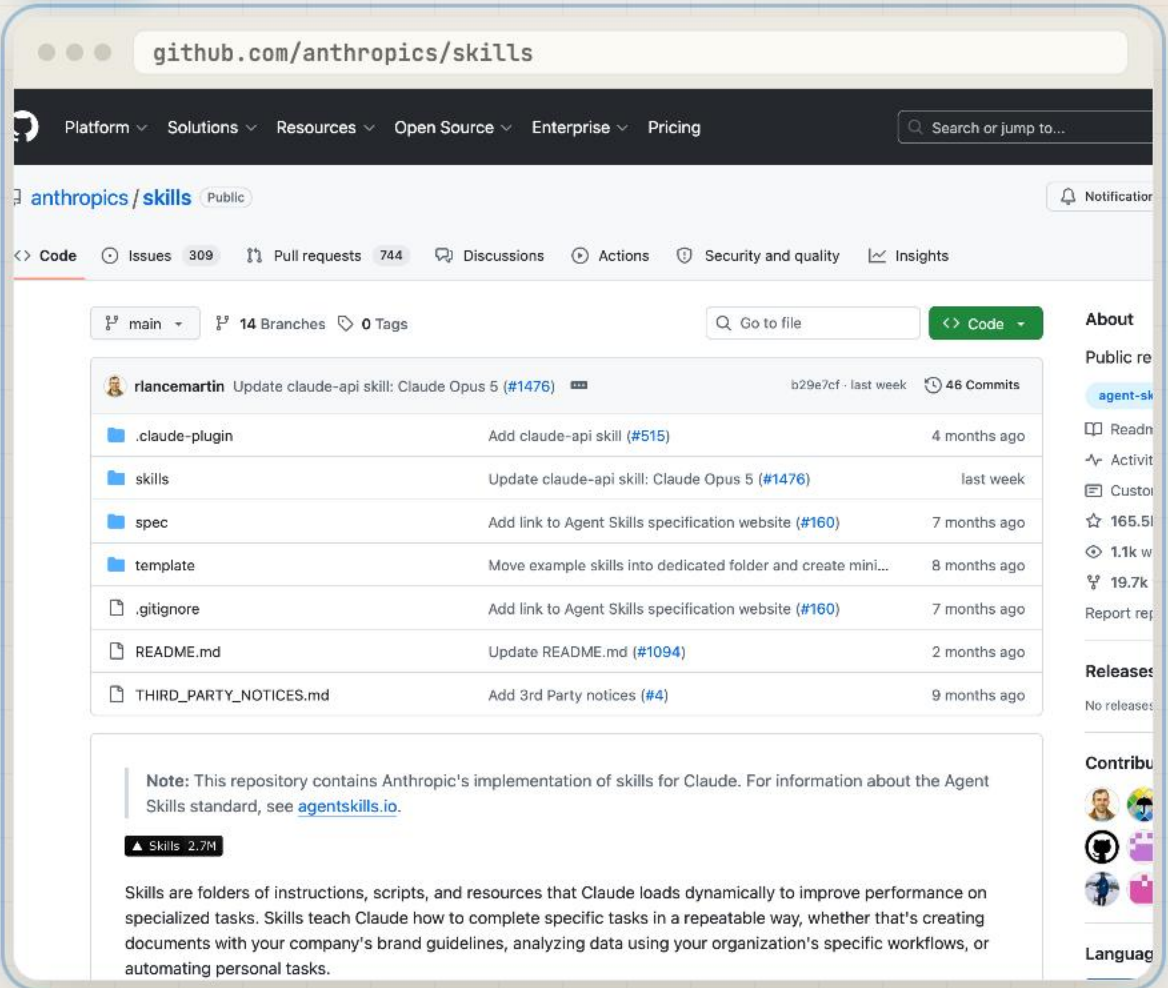


## THE COOKBOOKS

★ 165.5k

01

## Agent Skills



The screenshot shows the GitHub repository page for `anthropics/skills`. The repository is public and has 165.5k stars. The main branch is `main`, and there are 14 branches and 0 tags. The repository contains a list of files and folders, including `.claude-plugin`, `skills`, `spec`, `template`, `.gitignore`, `README.md`, and `THIRD_PARTY_NOTICES.md`. A note at the bottom states: "Note: This repository contains Anthropic's implementation of skills for Claude. For information about the Agent Skills standard, see [agentskills.io](https://agentskills.io)." The repository also has a "Skills" badge with a value of 2.7M.

**WHAT IT IS** Anthropic's own skills, including the ones that build Word, Excel, PowerPoint and PDF files.

**START HERE**

Clone the repo, drop one folder into your skills directory, Claude uses it on the next prompt.

## THE COOKBOOKS

★ 50.8k

## 02

## Claude Cookbooks

github.com/anthropics/claude-cookbooks

Platform Solutions Resources Open Source Enterprise Pricing Search or jump to...

anthropics / claude-cookbooks Public

<> Code Issues 73 Pull requests 220 Actions Projects Security and quality Insights

main 146 Branches 0 Tags Go to file Code

| Commit                                                        | Message                                                            | Time          |
|---------------------------------------------------------------|--------------------------------------------------------------------|---------------|
| Merge pull request #793 from anthropics/cj-ant/crop-tool-zoom | 85016ca · last week 612 Commits                                    |               |
| .claude                                                       | Add coordinator-pattern cost cookbook: big models plan, ...        | 29 days ago   |
| .github                                                       | CI: fail the notebook-tests job when a changed notebook f...       | 3 weeks ago   |
| anthropic_cookbook                                            | refactor: simplify notebook CI/CD by removing nbqa and p...        | 10 months ago |
| capabilities                                                  | fix(rag,classification): update rate-limit tier references to u... | 29 days ago   |
| claude_agent_sdk                                              | hosting: address #677 review comments                              | 2 months ago  |
| coding                                                        | docs: update all model references from Claude 4.5 to Clau...       | 5 months ago  |
| evals/agentic_search                                          | Reproduce Claude's agentic search benchmark scores in t...         | last month    |
| extended_thinking                                             | docs: update all model references from Claude 4.5 to Clau...       | 5 months ago  |
| fable_5_fallback_billing                                      | Adjust directory                                                   | last month    |
| finetuning                                                    | Lint + Format all cookbooks/scripts (#305)                         | 8 months ago  |
| images                                                        | Merge pull request #793 from anthropics/cj-ant/crop-tool-...       | last week     |
| managed_agents                                                | Merge pull request #759 from mongodb-partners/cma_wit...           | last week     |
| misc                                                          | Clean up metaprompt notebook: clear outputs, rename CL...          | 3 weeks ago   |

**About**  
A collect...  
showcas...  
ways of t...  
Readm...  
MIT lic...  
Contri...  
Activit...  
Custom...  
50.8k...  
618 w...  
6.0k f...  
Report re...

**Releases**  
No releases

**Contribu**

**WHAT IT IS** Ready-made recipes for classification, summarising, RAG, vision, tool use and prompt caching.

**START HERE**

Open the notebook closest to your job and change the prompt at the top. Nothing else.

03

# Claude Quickstarts

github.com/anthropics/claude-quickstarts

Platform Solutions Resources Open Source Enterprise Pricing

Search or jump to...

anthropics / claude-quickstarts Public

Notifications

<> Code Issues 115 Pull requests 85 Actions Projects Security and quality Insights

main 12 Branches 0 Tags

Go to file Code

cj-ant

Add Managed Agents Chat SDK quickstart (#437)

370e18d · 2 weeks ago

86 Commits

|                             |                                                             |               |
|-----------------------------|-------------------------------------------------------------|---------------|
| .github                     | Update to Claude API naming conventions (#293)              | 10 months ago |
| agents                      | feat: Add Claude Haiku 4.5 model support across all demo... | 9 months ago  |
| autonomous-coding           | Introduce Claude-Builds-Claude quickstart (#314)            | 8 months ago  |
| browser-use-demo            | Add computer-use-best-practices quickstart (#402)           | 2 months ago  |
| computer-use-best-practices | Add computer-use-best-practices quickstart (#402)           | 2 months ago  |
| computer-use-demo           | Add adaptive thinking support to the computer-use demo ...  | 2 months ago  |
| customer-support-agent      | feat: Add Claude Haiku 4.5 model support across all demo... | 9 months ago  |
| financial-data-analyst      | feat: Add Claude Haiku 4.5 model support across all demo... | 9 months ago  |
| managed-agents              | Add Managed Agents Chat SDK quickstart (#437)               | 2 weeks ago   |
| .pre-commit-config.yaml     | re-add precommit (#37)                                      | 2 years ago   |
| CLAUDE.md                   | Add Managed Agents Chat SDK quickstart (#437)               | 2 weeks ago   |
| LICENSE                     | customer support agent                                      | 2 years ago   |

About

A collect help dev with built using the

Readn

MIT lic

Activit

Custom

17.3k

166 w

3.0k f

Report rep

Releases

No releases

Contribu

**WHAT IT IS** Seven complete apps you can run: customer support agent, financial analyst, browser automation, autonomous coder.

**START HERE**

Start with the customer support agent. It runs locally in about ten minutes.



## THE COOKBOOKS

## 04

## The Metaprompt

The screenshot shows a Google Colab notebook interface. At the top, the URL bar displays `colab.research.google.com/.../metaprompt.ipynb`. Below the URL bar, the notebook title is `metaprompt.ipynb`. The menu bar includes `Edit`, `View`, `Insert`, `Runtime`, `Tools`, and `Help`. The toolbar shows `Commands`, `+ Code`, `+ Text`, `Run all`, and `Copy to Drive`. The notebook content is organized into sections:

- Metaprompt**

Welcome to the Metaprompt! This is a prompt engineering tool designed to solve the "blank page problem" and give you a starting point for iteration. All you need to do is enter your task, and optionally the names of the variables you'd like Claude to use in the template. Then you'll be able to run the prompt that comes out on any examples you like.

**Caveats**

  - This is designed for single-turn question/response prompts, not multiturn.
  - The prompt you'll get at the end is not guaranteed to be optimal by any means, so don't be afraid to change it!
- Using This Notebook**

The notebook is designed to be maximally easy to use. You don't have to write any code. Just follow these steps:

  - Run the first code cell to install the `anthropic` package if you don't already have it.
  - Enter your Claude API key in between quotation marks where it says "Put your API key here!" (If you're running outside Colab, you can instead leave it blank and set the `ANTHROPIC_API_KEY` environment variable. Remember to remove your key before sharing the notebook.)
  - Enter your task where it says "Replace with your task!"
  - Optionally, enter an all-caps list of variables in quotes separated by commas where it says "specify the input variables you want Claude to use".
  - If you changed the task, also put example values for your variables in `VARIABLE_VALUES` in the "Testing your prompt template" section near the bottom.

Then run all cells (Runtime -> Run all in Colab). Your generated prompt template appears partway down, in section 1, and the notebook finishes by test-driving it on your example values so you can see it working.

At the bottom of the notebook, a code cell is visible with the following content:

```
%%capture
# Install the anthropic package (a quick no-op if it's already up to date)
```

**WHAT IT IS** No idea how to start a prompt? This one writes your first draft for you.

**START HERE**

Paste your rough idea into the top cell, run it, keep the draft it hands back.

## 05

## Prompting best practices

The screenshot shows a web browser window with the address bar displaying `platform.claude.com/docs/.../claude-prompting-best-practices`. The page title is "Prompting best practices" and it includes a "Copy page" button. The content is organized into sections: "Best practices / Prompt engineering", "Comprehensive guide to prompt engineering techniques for Claude's latest models, covering clarity, examples, XML structuring, thinking, and agentic systems.", "This is the reference for prompt engineering with Claude's latest models, including Claude Fable 5, Claude Mythos 5, Claude Opus 5, Claude Opus 4.8, Claude Opus 4.7, Claude Opus 4.6, Claude Sonnet 5, Claude Sonnet 4.6, and Claude Haiku 4.5. The page is organized in three parts:", and a list of three parts: "Model-specific guidance", "Techniques for all current models", and "Migration considerations". A callout box provides additional links for model capabilities and migration guidance. The page also includes sections for "Claude Fable 5" and "Claude Sonnet 5" with their respective prompting guidance.

**WHAT IT IS** The living reference. Clarity, examples, XML, roles, thinking, tool use, agents.

**START HERE**

Read it once end to end, then keep the tab open the next time you write a real prompt.

## 06

## Prompting Fable 5

platform.claude.com/docs/.../prompting-claude-fable-5

Messages

Managed Agents

Admin

Resources ▾

() API reference



English ▾

Console

Best practices / Prompt engineering

## Prompting Claude Fable 5

Copy page ▾

Behavioral differences and prompting patterns for Claude Fable 5 and Claude Mythos 5, covering effort, instruction following, long runs, memory, and scaffolding changes.

This guide covers the prompting and scaffolding patterns specific to Claude Fable 5 and Claude Mythos 5. For the model's capabilities, API changes, pricing, and availability, see [Introducing Claude Fable 5 and Claude Mythos 5](#). For techniques that apply across all current Claude models, see [Prompting best practices](#).

Claude Fable 5 takes on problems that were previously too complex, long-running, or ambiguous for prior models, and is particularly effective at end-to-end work that takes a person hours, days, or weeks to complete. The teams seeing the best outcomes apply Claude Fable 5 to their hardest unsolved problems; testing it only on simpler workloads tends to undersell its capability range. It also performs reliably on more straightforward tasks.

Claude Fable 5 has several behavioral differences from Claude Opus 4.8 that may require prompt or scaffolding updates. Capability improvements at this level are also a good prompt to re-evaluate which instructions, tools, and guardrails are still needed. The patterns below cover the behaviors that most often require tuning.

ⓘ For API parameter changes specific to Claude Fable 5 and Claude Mythos 5 (adaptive thinking only, summarized-only thinking output, no extended thinking budgets, the [refusal](#) stop reason and fallback handling), see [Introducing Claude Fable 5 and Claude Mythos 5](#).

Claude Fable 5 runs safety classifiers that target offensive cybersecurity techniques (such as building exploits, malware, or attack tooling), biology and life sciences content (such as lab methods or molecular mechanisms), and extraction of the model's summarized thinking. Benign cybersecurity work and beneficial life sciences tasks may also trigger these safeguards. To re-route declined requests automatically, configure [server-side](#) or [client-side fallback](#) to Claude Opus 4.8.

## Capability improvements

Compared with Claude Opus 4.8, Claude Fable 5 shows improvement in:

- **Long-horizon autonomy.** Claude Fable 5 sustains productive output over extended periods, completing multiday, goal-

**WHAT IT IS** The most capable model Anthropic has released, and what to change in your prompts for it.

## START HERE

Read it before you hand Fable your hardest job. It prompts differently to the rest.



07

# Prompting Opus 5

platform.claude.com/docs/.../prompting-claude-opus-5

Messages

Managed Agents

Admin

Resources

( ) API reference

English

Console

Best practices / Prompt engineering

## Prompting Claude Opus 5

Copy page

Behavioral differences and prompting patterns for Claude Opus 5, covering response verbosity, agentic narration, task scoping, subagent delegation, self-correction, and output artifacts when thinking is disabled.

This guide covers the prompting patterns specific to Claude Opus 5. For the model's capabilities and API changes, see [What's new in Claude Opus 5](#). For techniques that apply across all current Claude models, see [Prompting best practices](#).

Claude Opus 5 is built for complex agentic coding and enterprise work, with particular strengths in long-horizon agentic tasks. It performs well out of the box on existing Claude Opus 4.8 prompts. The following patterns cover the behaviors that most often require tuning.

For API changes when migrating from Claude Opus 4.8 (thinking on by default, and disabling thinking capped at high effort), see the [migration guide](#).

## Capability improvements

Compared with Claude Opus 4.8, the improvements most relevant to prompting are:

- Agentic coding:** Claude Opus 5 is strongest on difficult coding tasks: multi-file features, larger refactors, and end-to-end feature work. It completes full tasks rather than leaving stubs or placeholders, and it performs best when given the complete task specification up front and left to run. It also performs well on easier tasks like single-turn edits, where the difference from prior models is smaller.
- Code review and bug-finding:** Claude Opus 5 reviews code with high precision and recall: it finds real bugs at a high rate per pass, and its additional findings are mostly real issues rather than false positives. Accuracy holds at lower effort settings, which supports a fast pass at review time and a more thorough pass later. If your review prompt says "only report high-severity issues" or "be conservative," the model may follow that instruction literally and report less; ask it to report everything and filter in a separate pass instead.
- Efficiency at lower effort:** low and medium effort produce strong quality at a fraction of the tokens and latency of higher settings. Start with the default ( high ) and adjust based on your evals: use low and medium liberally as your

**WHAT IT IS** The model Anthropic tells you to start on. Built for agentic work and heavy lifting.

### START HERE

Match your prompts to this page before blaming the model for a weak answer.



## THE PROMPTING GUIDES

08

## Prompting Sonnet 5

platform.claude.com/docs/.../prompting-claude-sonnet-5

Messages

Managed Agents

Admin

Resources ▾

() API reference



English ▾

Console

Best practices / Prompt engineering

## Prompting Claude Sonnet 5

Copy page ▾

Behavioral differences and prompting patterns for Claude Sonnet 5, covering effort, adaptive thinking defaults, tool use, and migration from Claude Sonnet 4.6.

This guide covers the prompting patterns specific to Claude Sonnet 5. For the model's capabilities and API changes, see [What's new in Claude Sonnet 5](#). For techniques that apply across all current Claude models, see [Prompting best practices](#).

Claude Sonnet 5 has particular strengths in coding and agentic tasks. It performs well out of the box on existing Claude Sonnet 4.6 prompts. The patterns in this guide cover the behaviors that most often require tuning.

For API parameter changes when migrating from Claude Sonnet 4.6 (adaptive thinking on by default, sampling parameters not accepted, manual extended thinking removed, and the new tokenizer), see the [migration guide](#).

## Response length and verbosity

Claude Sonnet 5 calibrates response length to the complexity of the task rather than defaulting to a fixed verbosity. This usually means shorter answers on simple lookups and longer ones on open-ended analysis.

If your product depends on a certain style or verbosity of output, you may need to tune your prompts. As an example, to decrease verbosity, you might add:

Provide concise, focused responses. Skip non-essential context, and keep examples minimal.

If you see specific kinds of verbosity (such as over-explaining), you can add additional instructions in your prompt to prevent them. Positive examples showing how Claude can communicate with the appropriate level of concision tend to be more effective than negative examples or instructions that tell the model what not to do.

## Calibrating effort and thinking depth

The effort parameter allows you to tune Claude's intelligence versus token spend, trading off capability for faster speed and

Ask Dr

**WHAT IT IS** The speed-and-brains pick, and where most of your daily work should sit.

**START HERE**

Move your everyday prompts here first. You will notice the speed before the cost.

8

9

10

11

12

13

14

15

16

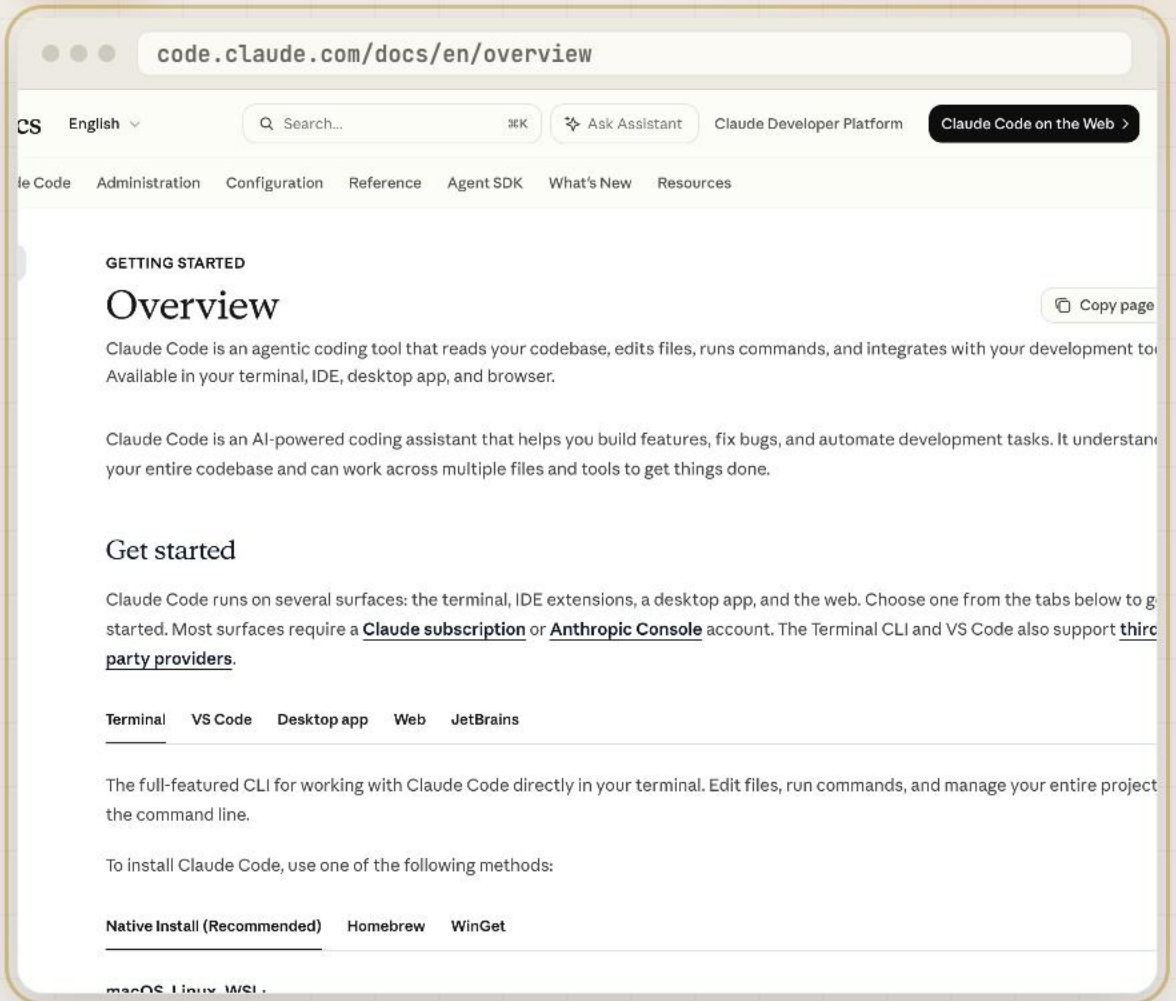
17

CH

CLAUDE CODE

09

# Claude Code docs



**WHAT IT IS** The full manual for the version of Claude that runs in your terminal.

**START HERE**

Skim the overview, then run `/init` inside a project folder and read what it writes.

## CLAUDE CODE

10

## Claude Code best practices

The screenshot shows the Claude Code documentation page for best practices. The URL in the browser is `code.claude.com/docs/en/best-practices`. The page has a navigation bar with links for 'le Code', 'Administration', 'Configuration', 'Reference', 'Agent SDK', 'What's New', and 'Resources'. There is a search bar, a 'Ask Assistant' button, and a 'Claude Developer Platform' link. A 'Copy page' button is visible on the right. The main content area is titled 'USE CLAUDE CODE' and 'Best practices for Claude Code'. It includes a sub-header 'Tips and patterns for getting the most out of Claude Code, from configuring your environment to scaling across parallel sessions.' The text describes Claude Code as an agentic coding environment that can read files, run commands, make changes, and autonomously work through problems. It mentions that this changes how you work, as you describe what you want and Claude figures out how to build it. It also notes that this autonomy comes with a learning curve and that Claude works within certain constraints. The guide covers patterns that have proven effective across Anthropic's internal teams and for engineers using Claude Code across various codebases, languages, and environments. It references a 'How Claude Code works' document. The text continues with 'Most best practices are based on one constraint: Claude's context window fills up fast, and performance degrades as it fills.' and 'Claude's context window holds your entire conversation, including every message, every file Claude reads, and every command output. However, this can fill up fast. A single debugging session or codebase exploration might generate and consume tens of thousands of tokens.' It concludes with 'This matters since LLM performance degrades as context fills. When the context window is getting full, Claude may start "forgetting" earlier instructions or making more mistakes. The context window is the most important resource to manage. To see how a session fills up in practice, [watch an interactive walkthrough](#) of what loads at startup and what each file read costs. Track context usage continuously.'

**WHAT IT IS** Verify-your-work loops, plan mode, CLAUDE.md, subagents and parallel sessions.

**START HERE**

Copy the CLAUDE.md pattern into your own project first. It changes every session after it.



CLAUDE CODE

11

# Agent SDK

code.claude.com/docs/en/agent-sdk/overview

CS English

Search...

Ask Assistant

Claude Developer Platform

Claude Code on the Web

Claude Code

Administration

Configuration

Reference

Agent SDK

What's New

Resources

AGENT SDK

Agent SDK overview

Build production AI agents with Claude Code as a library

An agent is an application that completes a task by planning its own steps and calling tools that read files, run commands, or edit code. Agent SDK gives you the same tools, agent loop, and context management that power Claude Code, programmable in Python and TypeScript.

Compare the Agent SDK to other Claude tools

The Agent SDK, the CLI, the Client SDK, and Managed Agents each fit different needs. Use the table to find the one that matches what you're building.

| If you're...                                                                                            | Use                    | Why                                                                                                  |
|---------------------------------------------------------------------------------------------------------|------------------------|------------------------------------------------------------------------------------------------------|
| Building an agent without implementing the tool loop yourself                                           | <u>Agent SDK</u>       | A library that runs the agent loop in your own process, in Python or TypeScript.                     |
| Doing interactive development or running one-off tasks from a terminal                                  | <u>Claude Code CLI</u> | The terminal interface, built for daily interactive use.                                             |
| Calling the API directly and implementing the tool loop yourself                                        | <u>Client SDK</u>      | Direct access to the Anthropic API rather than to Claude Code. You implement the tool loop yourself. |
| Running long-running or asynchronous agents without managing your own sandbox or session infrastructure | <u>Managed Agents</u>  | Hosted REST API, a separate product from the Agent SDK. Anthropic runs the agent and the sandbox.    |

**WHAT IT IS** Build your own agents on the same harness Claude Code runs on.

**START HERE**

Read the overview to see what an agent actually needs before you try to build one.

CLAUDE CODE

12

# MCP docs

modelcontextprotocol.io

version 2026-07-28 (latest) Search... Ask Assistant Blog GitHub

Extensions Registry SEPs Community

## Get started

### What is the Model Context Protocol (MCP)?

MCP (Model Context Protocol) is an open-source standard for connecting AI applications to external systems.

Using MCP, AI applications like Claude or ChatGPT can connect to data sources (e.g. local files, databases), tools (e.g. search engine calculators) and workflows (e.g. specialized prompts)—enabling them to access key information and perform tasks.

Think of MCP like a USB-C port for AI applications. Just as USB-C provides a standardized way to connect electronic devices, MCP provides a standardized way to connect AI applications to external systems.

```
graph LR; subgraph AI_applications [AI applications]; direction TB; A1[Chat interface  
Claude Desktop, LibreChat]; A2[IDEs and code editors  
Claude Code, Goose]; A3[Other AI applications  
Sire, Superinterface]; end; subgraph Data_sources_and_tools [Data sources and tools]; direction TB; D1[Data and file systems  
PostgreSQL, SQLite, GDrive]; D2[Development tools  
Git, Sentry, etc.]; D3[Productivity tools  
Slack, Google Maps, etc.]; end; MCP[MCP  
Standardized protocol]; A1 <--> MCP; A2 <--> MCP; A3 <--> MCP; MCP <--> D1; MCP <--> D2; MCP <--> D3;
```

**WHAT IT IS** Wire Claude into your own tools and data, so it reads live instead of guessing.

**START HERE**

Connect one tool you already live in, Notion or Drive, and stop there for a week.

THE COURSES

★ 37.5k

13

# Interactive tutorial

**WHAT IT IS** Nine chapters, beginner to advanced, with exercises you actually do.

**START HERE**

Do chapters one and two today. They take about twenty minutes together.



## THE COURSES

★ 22.5k

14

## Anthropic Courses

The screenshot shows the GitHub repository page for `anthropics/courses`. The repository is public and has 17 branches and 0 tags. It contains five folders: `anthropic_api_fundamentals`, `prompt_engineering_interactive_tutorial`, `prompt_evaluations`, `real_world_prompting`, and `tool_use`. There are also files for `.gitignore`, `LICENSE`, and `README.md`. The repository has 59 commits and was last updated 8 months ago. The repository description states: "Welcome to Anthropic's educational courses. This repository currently contains five courses. We suggest completing the courses in the following order: 1. [Anthropic API fundamentals](#) - teaches the essentials of working with the Claude SDK: getting an API key, working with model parameters, writing multimodal prompts, streaming responses, etc. 2. [Prompt engineering interactive tutorial](#) - a comprehensive step-by-step guide to key prompting techniques. [AWS Workshop version]"

**WHAT IT IS** Five full courses: API fundamentals, prompting, real-world prompting, evaluations, tool use.

**START HERE**

Start on the prompting course. Skip the API one unless you write code.

14

15

16

17

CH

THE COURSES

15

# Anthropic Academy



anthropic.skilljar.com

Anthropic



through a chat window.



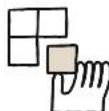
## Introduction to Claude Cowork

Learn to work alongside Claude on your real files and projects. This hands-on course covers the Cowork task loop, plugins and skills, file and research workflows, and how to steer multi-step work responsibly — so you're productive in your first week.



## Claude Code in Action

Run long, hands-off Claude Code sessions you can trust: steer, configure, automate, and verify



## AI Fluency: Framework & Foundations

Learn to collaborate with AI systems effectively, efficiently, ethically, and safely

**WHAT IT IS** Anthropic's own course platform, including Claude Code in Action, agent skills and MCP.

### START HERE

Sign in and take Claude Code in Action first. It is the one that changes how you work.

## THE ENGINEERING WRITE-UPS

16

# Building effective agents

[anthropic.com/engineering/building-effective-agents](https://anthropic.com/engineering/building-effective-agents)[Research](#) [Policy](#) [Commitments](#) [Learn](#) [News](#)[Try Claude](#)

## Building effective agents

We've worked with dozens of teams building LLM agents across industries. Consistently, the most successful implementations use simple, composable patterns rather than complex frameworks.

Over the past year, we've worked with dozens of teams building large language model (LLM) agents across industries. Consistently, the most successful implementations weren't using complex frameworks or specialized libraries. Instead, they were building with simple, composable patterns.

In this post, we share what we've learned from working with our customers and

**WHAT IT IS** The five workflow patterns, named and drawn. Prompt chaining, routing, parallelisation, orchestrator-workers, evaluator-optimiser.

**START HERE**

Read the five and name the one your current workflow already is.



## THE ENGINEERING WRITE-UPS

17

## Context engineering

[anthropic.com/engineering/effective-context-engineering-for-ai-agents](https://anthropic.com/engineering/effective-context-engineering-for-ai-agents)

Research Policy Commitments ▾ Learn ▾ News

Try Claude ▾

## Effective context engineering for AI agents

Context is a critical but finite resource for AI agents. In this post, we explore strategies for effectively curating and managing the context that powers them.

After a few years of prompt engineering being the focus of attention in applied AI, a new term has come to prominence: **context engineering**. Building with language models is becoming less about finding the right words and phrases for your prompts, and more about answering the broader question of “what configuration of context is most likely to generate our model’s desired behavior?”

**Context** refers to the set of tokens included when sampling from a large-language model (LLM). The **engineering** problem at hand is optimizing the utility of those tokens against the inherent constraints of LLMs in order to consistently achieve a desired outcome. Effectively wrangling LLMs often

**WHAT IT IS** Why long chats get worse, and the three fixes: compaction, notes, sub-agents.

**START HERE**

Apply the three fixes the next time a long chat starts drifting off the brief.